

Awantika AIparc 1

March 30, 2026

```
[1]: from collections import deque

# Graph representation
graph = {
    0: [1, 2],
    1: [0, 3, 4],
    2: [0, 5],
    3: [1],
    4: [1, 5],
    5: [2, 4]
}

# 1. Recursive DFS

def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()

    print(node, end=" ")
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

print("DFS Recursive:")
dfs_recursive(graph, 0)
```

DFS Recursive:

0 1 3 4 5 2

```
[2]: # 2. Non-Recursive DFS

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
```

```

while stack:
    node = stack.pop()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        # reverse to maintain order
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)

print("\n\nDFS Non-Recursive:")
dfs_iterative(graph, 0)

```

DFS Non-Recursive:
0 1 3 4 5 2

[3]: # 3. Non-Recursive BFS

```

def bfs_iterative(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

print("\n\nBFS Non-Recursive:")
bfs_iterative(graph, 0)

```

BFS Non-Recursive:
0 1 2 3 4 5

```
[4]: # 4. Recursive BFS

def bfs_recursive_helper(queue, visited, graph):
    if not queue:
        return

    node = queue.popleft()
    print(node, end=" ")

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append(neighbor)

    bfs_recursive_helper(queue, visited, graph)

def bfs_recursive(graph, start):
    visited = set([start])
    queue = deque([start])
    bfs_recursive_helper(queue, visited, graph)

print("\n\nBFS Recursive:")
bfs_iterative(graph, 0)
```

BFS Recursive:

0 1 2 3 4 5